MiniKV: um banco chave-valor transacional

Bancos de dados como PostgreSQL, MySQL e Redis atendem milhares de clientes ao mesmo tempo. Para isso, precisam resolver três problemas clássicos de programação concorrente:

1. Permitir que muitos clientes acessem os dados **em paralelo** sem corromper nada;
2. Garantir que uma operação confirmada ao cliente **não se perca** se a máquina desligar logo depois;
3. Conseguir tirar uma **cópia consistente** dos dados (um *snapshot*) sem parar o banco por muito tempo.

Neste trabalho, você vai implementar a sincronização de um pequeno banco chave-valor em memória, o **MiniKV**, que guarda saldos de contas bancárias. A estrutura de dados (a tabela hash), a escrita em disco, os clientes e a verificação de corretude **já estão prontos** no código base. Seu trabalho é fazer tudo isso funcionar corretamente com muitas threads.

Visão Geral

O MiniKV guarda `A` contas (opção `-a`), identificadas por chaves do tipo `"conta-0042"`. Cada conta começa com o mesmo saldo inicial. O banco é acessado concorrentemente por:

- `C` **threads cliente** (opção `-c`): cada cliente executa `N` operações aleatórias (opção `-n`). Cada operação é uma **consulta de saldo** ou uma **transferência** entre duas contas. O código dos clientes já está pronto e não deve ser alterado.
- **Uma thread de log** (*log writer*): grava as transferências em disco (veja a Parte 2).
- **Uma thread de snapshot**: periodicamente tira uma cópia de todos os saldos (veja a Parte 3).

Como transferências só movem dinheiro de uma conta para outra, **a soma de todos os saldos nunca muda**. Esse é o principal invariante do banco, e o programa o verifica ao final da execução.

O trabalho é dividido em **quatro partes**, descritas nas seções a seguir:

Parte	O que implementar	Arquivo
1. Acesso às Contas	Sincronização das consultas e transferências, por partição	`src/db.c`
2. Durabilidade	O log com *group commit*: buffer de log e thread de log	`src/wal.c`
3. Snapshots	Cópia consistente de todos os saldos, sem *starvation*	`src/db.c`
4. Inicialização e Término	Criação das threads e encerramento ordenado do servidor	`src/server.c`

As partes não são independentes: as transferências da Parte 1 também devem respeitar os snapshots da Parte 3, e o término da Parte 4 depende de como o log da Parte 2 foi implementado.

Parte 1: Acesso às Contas

A tabela hash que guarda as contas é dividida em `S` **partições** (opção `-s`). A conta com chave `k` pertence à partição `hash(k) % S`. A função `db_partition_of(...)`, já pronta, calcula a partição de uma chave.

As partições **não** são estruturas de dados separadas: a tabela continua sendo uma só (`table_t`), e cada conta simplesmente "pertence" a uma partição. As partições existem para a **sincronização**: cada partição deve ter a sua própria proteção, e o acesso a uma conta é controlado pela proteção da partição em que ela está. Assim, operações sobre contas de partições diferentes não disputam a mesma trava e podem executar em paralelo.

O número de partições define a **granularidade** da sincronização. Com `-s 1` , todas as contas ficam na mesma partição e as operações se comportam como se houvesse uma única trava para o banco inteiro. Com mais partições, há menos disputa entre as threads e mais paralelismo.

As duas operações do banco já estão prontas no código base e chamam funções de sincronização que você deve implementar:

```
int db_get(db_t *self, const char *key)          /* JÁ PRONTA */
{
    db_acquire_read(self, key);                  /* VOCÊ IMPLEMENTA */
    int balance = table_get(self->table, key);
    db_release_read(self, key);                  /* VOCÊ IMPLEMENTA */
    return balance;
}

long db_transfer(db_t *self, const char *from, const char *to, int amount)   /* JÁ PRONTA */
{
    long lsn = -1;
    db_acquire_pair(self, from, to);             /* VOCÊ IMPLEMENTA */
    if (table_get(self->table, from) >= amount) {
        table_add(self->table, from, -amount);
        table_add(self->table, to, amount);
        lsn = wal_append(self->wal, from, to, amount);   /* VOCÊ IMPLEMENTA */
    }
    db_release_pair(self, from, to);             /* VOCÊ IMPLEMENTA */
    if (lsn >= 0)
        wal_wait_durable(self->wal, lsn);        /* VOCÊ IMPLEMENTA */
    return lsn;
}
```

- `db_acquire_read(...)` **/** `db_release_read(...)` : protegem uma consulta de saldo. Várias consultas a uma mesma partição podem acontecer **ao mesmo tempo**, mas nenhuma consulta pode acontecer enquanto uma transferência modifica aquela partição.
- `db_acquire_pair(...)` **/** `db_release_pair(...)` : protegem as duas contas de uma transferência. A transferência deve ser **atômica**: nenhuma outra thread pode ver o dinheiro "saindo" de uma conta sem "chegar" na outra.

Regras:

- Não é permitido usar um único lock para o banco inteiro: operações em partições diferentes devem poder executar em paralelo.
- Uma transferência envolve duas contas, que podem estar em partições diferentes ou na mesma partição. A solução não pode ter **deadlock** em nenhum desses casos.
- Saldos nunca podem ficar negativos.

Parte 2: Durabilidade (o Log de Escrita Antecipada)

Em um banco de dados real, uma transferência só pode ser confirmada ao cliente depois de estar gravada em disco. Assim, se o servidor desligar logo depois, a operação não se perde. Para isso, cada transferência aceita gera um **registro de log**, que é gravado em um arquivo (*write-ahead log*, ou WAL).

O log é um arquivo em que os registros são sempre **acrescentados no final**, nunca alterados. O nome *write-ahead* ("escrita antecipada") vem da regra de que a operação é anotada no log **antes** de ser confirmada ao cliente. Se o servidor cair, basta reler o log do início ao fim e reaplicar as transferências para reconstruir os saldos. Ao final da execução, o programa faz exatamente essa reconstrução e a compara com os saldos em memória. Por isso, a ordem dos registros no log deve corresponder à ordem em que as transferências foram aplicadas.

Cada registro é identificado por um **LSN** (*log sequence number*), a sua posição no log. Dizemos que um registro está **durável** quando ele já foi gravado em disco. Um cliente só pode ser liberado depois que o registro da sua transferência estiver durável.

Gravar em disco com `fsync` é lento (opção `-f`, em milissegundos). Por isso, gravar um registro de cada vez seria um desperdício. A solução usada por bancos reais é o ***group commit***: os clientes colocam seus registros em um **buffer de log** e uma única thread grava **todos os registros acumulados de uma só vez**.

O funcionamento esperado é o seguinte:

1. Ao aceitar uma transferência, `db_transfer(...)` coloca o registro no buffer de log com `wal_append(...)`. Essa função devolve o **LSN** (*log sequence number*) do registro, um número sequencial (0, 1, 2, ...) que identifica sua posição no log.
2. O buffer comporta no máximo `B` registros (opção `-b`). Se estiver cheio, `wal_append(...)` espera até haver espaço.
3. A thread de log (`wal_writer_run(...)`) retira **todos** os registros disponíveis no buffer e os grava de uma vez com a função `disk_write_batch(...)`, que já está pronta.
4. Depois de liberar as contas, `db_transfer(...)` chama `wal_wait_durable(...)`, que só retorna quando o registro com aquele LSN já foi gravado em disco.

Regras:

- Os registros devem ser gravados em disco **na ordem dos LSNs**, sem lacunas nem repetições.
- Nenhum cliente pode ser liberado por `wal_wait_durable(...)` antes de seu registro estar em disco, e nenhum cliente pode ficar esperando um registro que já foi gravado.
- A thread de log não pode ficar em espera ocupada quando o buffer está vazio.

Observe no código de `db_transfer(...)` que `wal_append(...)` é chamada **com** as contas travadas e `wal_wait_durable(...)` **sem** elas. Entender por que o código foi escrito assim faz parte do trabalho.

Parte 3: Snapshots

A cada `P` milissegundos (opção `-p`), a thread de snapshot chama `db_snapshot(...)`, que já está pronta. Essa função copia os saldos de **todas** as contas para um vetor entre chamadas a `db_snapshot_begin(...)` e `db_snapshot_end(...)`, que você deve implementar.

O snapshot deve ser **consistente**: ele precisa representar um estado em que o banco realmente esteve. Na prática, nenhuma transferência pode ter sido copiada "pela metade" (com o débito de uma conta e

sem o crédito da outra). Para cada snapshot, o programa verifica se a soma dos saldos copiados é igual à soma inicial.

Regras:

- Enquanto o snapshot é tirado, as transferências podem ser pausadas. Fora isso, o banco deve funcionar normalmente.
- **O snapshot não pode sofrer starvation.** Com muitos clientes fazendo operações sem parar, o snapshot ainda deve conseguir executar em um tempo razoável. O programa mede e imprime o maior tempo de espera de um snapshot.

Parte 4: Inicialização e Término

As threads são criadas pelas funções `server_spawn_client(...)` (clientes) e `server_start(...)` (thread de log e thread de snapshot). O código base já cria as threads; complete essas funções se a sua solução precisar.

A função `server_shutdown(...)` é chamada pelo `main.c` depois que todos os clientes terminaram. Ela deve:

1. Garantir que todos os registros pendentes foram gravados em disco;
2. Encerrar a thread de log, inclusive se ela estiver bloqueada esperando registros;
3. Encerrar a thread de snapshot (a função `snapshot_stop(...)`, já pronta, faz isso);
4. Aguardar o término dessas threads.

O que Já Está Pronto e o que Implementar

Você só pode alterar os arquivos `db`, `wal` e `server` (`.c` e `.h`). Todos os demais estão prontos e não podem ser alterados:

Arquivos	Conteúdo	Alterar?
`src/db.c`	O banco: sincronização das contas e dos snapshots	**Sim**, exceto `db_partition_of`, `db_get`, `db_transfer` e `db_snapshot`, no final do arquivo
`src/wal.c`	O log com group commit e a thread de log	**Sim**
`src/server.c`	Criação e encerramento das threads	**Sim**, exceto `server_join_clients`
`include/db.h`, `include/wal.h`, `include/server.h`	Structs e assinaturas das funções acima	**Sim**, para adicionar atributos às structs e assinaturas de novas funções
`src/main.c`	Programa principal	Não
`src/config.c`, `include/config.h`	Argumentos de linha de comando, constantes e funções auxiliares	Não
`src/storage.c`, `include/storage.h`	Tabela hash de contas (sem nenhuma sincronização) e gravação em disco, incluindo a simulação do `fsync`	Não
`src/workload.c`, `include/workload.h`	Clientes, thread de snapshot e verificação de corretude	Não
`Makefile`, `compile_flags.txt`	Compilação e configuração do editor (clangd)	Não

Cada arquivo também começa com um comentário que descreve seu conteúdo e diz se ele deve ou não ser alterado.

Todas as contas são criadas na inicialização; depois disso, só os saldos mudam. A tabela hash às vezes cede o processador no meio de uma leitura ou escrita. Isso não muda o resultado de uma solução correta, mas faz as condições de corrida aparecerem com muito mais frequência.

Funções que você deve implementar:

Arquivo	Funções
`db.c`	`db_init`, `db_finalize`, `db_acquire_read`, `db_release_read`, `db_acquire_pair`, `db_release_pair`, `db_snapshot_begin`, `db_snapshot_end`
`wal.c`	`wal_init`, `wal_finalize`, `wal_append`, `wal_wait_durable`, `wal_writer_run`
`server.c`	`server_shutdown` (e complementos em `server_init`, `server_finalize`, `server_start`, `server_spawn_client`, se necessário)

Você também deve incluir nas structs `db_t`, `wal_t` e `server_t` os atributos necessários para implementar esses comportamentos. Essas structs simulam os atributos de uma classe no modelo de programação orientado a objetos e devem ser inicializadas na função `*_init(...)` correspondente. Se houver alocação dinâmica de memória, libere-a na função `*_finalize(...)` correspondente.

Você pode usar qualquer primitiva da biblioteca `pthread` ou de `semaphore.h` : mutexes, variáveis de condição, semáforos, `pthread_rwlock_t` e barreiras. Escolha a que fizer mais sentido em cada caso: você precisará justificar suas escolhas na apresentação.

Evite sempre que possível criar variáveis globais. Você também pode criar novas funções: faça isso para deixar sua solução mais elegante e legível.

Requisitos gerais:

- Não é permitido usar **espera ocupada** (laços que testam uma condição repetidamente, com ou sem `msleep`).
- A solução não pode ter condições de corrida, deadlocks ou starvation.

AVISO IMPORTANTE 1: Não é permitido alterar os arquivos marcados com "Não" na tabela acima, nem as funções indicadas como prontas em `db.c` e `server.c` ! Na correção, esses arquivos serão substituídos pelas versões originais.

AVISO IMPORTANTE 2: Para cada função adicionada, inclua sua assinatura como `extern` no header (`.h`) e sua implementação no arquivo `.c` correspondente.

Verificação

Ao final da execução, o programa imprime:

- A soma dos saldos em memória comparada com a soma inicial (`[SUM MISMATCH]`) e se algum saldo ficou negativo (`[NEGATIVE BALANCE]`);
- Quantas transferências foram confirmadas aos clientes antes de estarem gravadas em disco (`[NOT DURABLE]`);
- Estatísticas da thread de log: número de `fsync` s, tamanho médio dos lotes, registros fora de ordem (`[LOG ORDER]`), chamadas concorrentes a `disk_write_batch(...)` (`[CONCURRENT WRITE]`) e chamadas com lote vazio (`[BUSY WAITING?]`);
- O resultado da reconstrução dos saldos a partir do arquivo de log: transferências que não estão no log (`[LOST TRANSFER]`), saldos que ficariam negativos (`[NEGATIVE BALANCE]`) e contas cujo saldo reconstruído difere do saldo em memória (`[REPLAY MISMATCH]`);
- Quantos snapshots foram tirados, quantos foram inconsistentes (`[INCONSISTENT SNAPSHOT]`) e os tempos médio e máximo de espera de um snapshot;
- O número de operações por segundo.

Ao final, o programa informa o número de erros detectados e termina com código de saída diferente de zero se houver algum.

A ausência de mensagens de erro **não** garante que a solução está correta: várias condições de corrida só se manifestam em execuções específicas. Teste com diferentes parâmetros, por exemplo com poucas contas e muitos clientes (muita disputa) ou com uma única partição. Recomendamos usar o *ThreadSanitizer* e o Helgrind (`valgrind --tool=helgrind`). Para compilar com o ThreadSanitizer, execute `make clean && make ADDONS=-fsanitize=thread` .

Compilação e Execução

Para compilar o projeto, execute:

```
$ make
```

Para limpar completamente o projeto, execute:

```
$ make clean
```

Para testar um cenário com 16 clientes fazendo 500 operações cada, sobre 64 contas divididas em 8 partições, com buffer de log de 8 registros, `fsync` de 5 ms e snapshots a cada 50 ms (os valores padrão), execute:

```
$ ./program -c 16 -n 500 -a 64 -s 8 -b 8 -f 5 -p 50
```

Execute `./program -h` para ver todas as opções.

O desempenho não será avaliado por um tempo mínimo. Mesmo assim, uma solução correta deve se beneficiar de mais partições e de lotes maiores no log. Compare as estatísticas variando `-s`, `-b` e `-c`: os resultados poderão ser discutidos na apresentação. Observe que cada cliente tem no máximo uma transferência esperando o disco, então o buffer de log só enche quando `-b` é menor que `-c`.

DICA: Para adicionar informações de debug na saída do programa, compile com `RELEASE=false`. Por padrão, as informações de debug estão desabilitadas. Para habilitá-las, faça:

```
$ make clean && make RELEASE=false
```

Com `RELEASE` em `false`, o programa imprime informações da execução e é compilado com a opção de debug para o gdb (flag `-g`). Essas informações são geradas por chamadas à função `plog(...)` no código. Você também pode usar `plog(...)` para imprimir outras informações de debug.

Para gerar um arquivo compactado com a sua solução final, execute:

```
$ make submission
```

Esse comando cria o arquivo `solution.tar.gz` no diretório principal. Esse arquivo deve ser submetido via Moodle até o prazo final de entrega.

Apresentação e Avaliação

O trabalho será apresentado oralmente pelo grupo. Cada membro será arguido individualmente sobre qualquer parte da solução e poderá ser solicitado a explicar ou modificar o código durante a apresentação. A nota de cada membro considera seu desempenho na apresentação.

A avaliação será feita com base na solução entregue pelo grupo (S), composta pelo código-fonte e relatório, e na apresentação individual de cada aluno(a) ao professor (A_i). A nota atribuída à cada aluno(a) *i* no trabalho (NotaTrabalho_i) será calculada da seguinte forma:

$$\text{NotaTrabalho}_i = \frac{A_i \times S}{10}$$

ATENÇÃO: Como indicado pela fórmula mostrada acima, a nota atribuída à solução adotada será ponderada pelo desempenho do(a) aluno(a) durante a apresentação do trabalho. Por exemplo, se o professor atribuir nota 10 para a solução adotada pelo grupo (S=10,0) mas o(a) aluno(a) receber nota 5,0 pela apresentação, devido ao desconhecimento dos conteúdos teóricos, práticos e/ou da solução do trabalho, a sua nota final do trabalho será 5. A ausência no dia da apresentação ou recusa de realização da apresentação do trabalho implicará em nota zero na apresentação, fazendo com que a nota final atribuída ao(à) aluno(a) também seja zero.

A tabela abaixo apresenta os critérios e respectivos pesos que serão utilizados para avaliação da solução (S) desenvolvida pelo grupo:

Critério	**Peso**
Parte 1: Acesso às Contas	25%
Parte 2: Durabilidade (o Log)	40%
Parte 3: Snapshots	25%
Parte 4: Inicialização e Término	10%